Whole Pipeline – Steps

1, Setup & Data Preparation

get the images ready so the model can read them

1.1 Environment Setup

connect Google Drive (on Colab) and set plot settings

1.3 Download Dataset

download CIFAR-10 once, skip if it is already there

1.4 Load Dataset

read all images and split them into train , val and test

1.5 Prepare Data

flatten each image to one row and center it around zero

2, Model Creation & Initialization

build the network once, before training

2.1 Define Architecture

choose the layers and their order

2.2 Layer Dimensions

decide how many numbers go in and out of each layer

2.3 Initialize Parameters

give weights small random values and biases zero

3, Training Setup & Mini-Batch

prepare everything the training loop needs

3.1 Select Training Data

full data for real training , small data to test for overfitting

3.2 Training Hyperparameters

set learning rate , batch size and number of epochs

3.3 Create Solver

put model , data and settings together in one trainer

3.4 Start Mini-Batch ⟳

pick a small random group of images , the loop starts here

4, Forward Pass

push the images through the network to get scores

4.1 Prepare Input

take the batch and switch the model to training mode

4.2 Hidden Layer Forward

each hidden layer mixes , normalizes , filters and randomly drops values

4.3 Final Output Layer

last layer turns features into one score per class

4.4 Scores

the raw guess of the model for every image

5, Loss Calculation

measure how wrong the guesses are

5.1 Softmax

turn scores into probabilities that add up to one

5.2 Correct-Class Probabilities

keep only the probability given to the true class

5.3 Data Loss

low probability for the true class means a high loss

low probability for the true class means a high loss

5.4 Regularization Loss

add a penalty for weights that grow too large

5.5 Total Loss & dout

add the two losses and start the gradient for backward

6, Backward Pass & Gradient

find how much each parameter caused the error

6.1 Receive dout

take the error signal coming from the loss

6.2 Final Affine Backward

compute gradients for the last layer's weights and bias

6.3 Hidden Layers Backward

pass the error back through each hidden layer in reverse order

6.4 All Gradients Ready

every parameter now has its own gradient

7, Optimizer & Parameter Update

change the parameters to reduce the loss

7.1 Receive Current Parameters

take each parameter , its gradient and its saved settings

7.2 Choose Update Rule

pick how to step : SGD , Momentum , RMSProp or Adam

7.3 Compute Next Parameters

calculate the new value of every parameter

7.4 Update Model Parameters

save the new values and go back to 3.4

After each iteration ↩

Print Loss

show the loss every 100 iterations

End of Epoch

count the epoch and make the learning rate a bit smaller

Check Accuracy

test the model on train and val images

Save Best Model

keep the parameters with the highest val accuracy

Loop End

put the best saved parameters back into the model

8, Evaluation & Accuracy

see how good the final model is

8.1 Plot Accuracy

draw train and val accuracy for each epoch

8.2 Test Accuracy

run the model on test images it has never seen

8.3 Print Results

show the final test accuracy and the best val accuracy

Steps 4 → 7 by hand

run one iteration step by step to show what happens inside the loop